

程序设计 II 第 17 周 题目汇总（完整版 · STL 容器适配器+算法）

来源：Matrix · 课程「程序设计 II（25 级 曾坤 计算机学院）」含：完整题面、提交文件、平台提供的初始文件（.hpp/main.cpp 等）。注：本周作业 06-29 截止，平台**尚未公布理论题标准答案**，下方理论题为整理的参考答案。

共 7 项：课后 1₄、课堂 1₂、理论题（20 题）

25 程设II-周 17-课后 1

类型：编程题

题面

有效的括号

给定只包含 ()[]{} 的字符串 s，判断括号是否**有效**：① 类型匹配（左括号用相同类型右括号闭合）② 顺序正确（右括号闭合离它最近、还未闭合的左括号）。

实现 bool isValid(const string& s)：合法返回 true，否则 false，空字符串合法。提示用 std::stack<char>：左括号压栈，右括号检查栈顶匹配，最终栈空则成功。输入长度 0~200。

输入: ()[]{} 输出: true

提交文件：BracketMatch.cpp

平台提供的初始文件

文件 main.cpp：

```
#include <iostream>
#include "BracketMatch.hpp"
```

```
int main() {
    string s;
    getline(cin, s);

    if (isValid(s))
        cout << "true";
    else
        cout << "false";

    return 0;
}
```

文件 BracketMatch.hpp：

```
#ifndef BRACKETMATCH_H
#define BRACKETMATCH_H

#include <string>

using namespace std;

bool isValid(const string& s);

#endif
```

25 程设II-周 17-课后 2

类型：编程题

题面

银行排队叫号

实现 BankQueue（用 std::queue<int>）：arrive(int id)(push)、call()(front+pop 返回 id)、next()(front)、waiting()(size)、empty()。操作 1 id/2/3/4。

输入: 8 / 1 101 / 1 102 / 1 103 / 3 / 4 / 2 / 2 / 4

输出: 101 / 3 / 101 / 102 / 1

提交文件：BankQueue.cpp

平台提供的初始文件

文件 main.cpp：

```
#include <iostream>
#include "BankQueue.hpp"

int main() {
    BankQueue bq;
    int n;
    cin >> n;

    for (int i = 0; i < n; i++) {
        int op;
        cin >> op;
        if (op == 1) {
            int id;
            cin >> id;
            bq.arrive(id);
        } else if (op == 2) {
            cout << bq.call() << endl;
        } else if (op == 3) {
            int id;
            cin >> id;
            bq.next(id);
        } else if (op == 4) {
            cout << bq.empty() << endl;
        }
    }
}
```

```

        cout << bq.next() << endl;
    } else if (op == 4) {
        cout << bq.waiting() << endl;
    }
}

return 0;
}

```

文件 BankQueue.hpp :

```

#ifndef BANKQUEUE_H
#define BANKQUEUE_H

#include <queue>

using namespace std;

class BankQueue {
private:
    queue<int> q;

public:
    BankQueue();
    void arrive(int id);
    int call();
    int next();
    int waiting();
    bool empty();
};

#endif

```

25 程设II-周 17-课后 3

类型：编程题

题面

急诊分诊

实现 EmergencyRoom（用 priority_queue，大顶堆，越紧急越先，严重程度互不相同）：arrive(int id,int severity)(push)、treat()(top+pop 返回 id)、next()(top)、waiting()(size)。提示用 pair<int,int>（first 严重程度、second id，pair 先比 first 再比 second）。操作 1 id severity/2/3/4。

输入: 7 / 1 101 3 / 1 102 8 / 1 103 1 / 3 / 2 / 2 / 4

输出: 102 / 102 / 101 / 1

提交文件：EmergencyRoom.cpp

平台提供的初始文件

文件 main.cpp：

```
#include <iostream>
#include "EmergencyRoom.hpp"

int main() {
    EmergencyRoom er;
    int n;
    cin >> n;

    for (int i = 0; i < n; i++) {
        int op;
        cin >> op;
        if (op == 1) {
            int id, severity;
            cin >> id >> severity;
            er.arrive(id, severity);
        } else if (op == 2) {
            cout << er.treat() << endl;
        } else if (op == 3) {
            cout << er.next() << endl;
        } else if (op == 4) {
            cout << er.waiting() << endl;
        }
    }

    return 0;
}
```

文件 EmergencyRoom.hpp：

```
#ifndef EMERGENCYROOM_H
#define EMERGENCYROOM_H

#include <queue>

using namespace std;

class EmergencyRoom {
private:
    priority_queue<pair<int, int>> pq;

public:
    EmergencyRoom();
    void arrive(int id, int severity);
    int treat();
```

```
    int next();  
    int waiting();  
};
```

```
#endif
```

25 程设II-周 17-课后 4

类型：编程题

题面

成绩分析

实现三个函数：sortDesc(vector<int>& scores) (std::sort + lambda 降序) 、 countPass(const vector<int>& scores) (std::count_if + [](int x){return x>=60;}) 、 average(const vector<int>& scores) (std::accumulate，头文件 <numeric>，求和÷人数) 。

输入: 5 / 45 78 92 60 33

输出: 92 78 60 45 33 / 3 / 61.6

提交文件：ScoreAnalyzer.cpp

平台提供的初始文件

文件 main.cpp：

```
#include <iostream>  
#include <iomanip>  
#include "ScoreAnalyzer.hpp"  
  
int main() {  
    int n;  
    cin >> n;  
  
    vector<int> scores(n);  
    for (int i = 0; i < n; i++) {  
        cin >> scores[i];  
    }  
  
    sortDesc(scores);  
    for (int i = 0; i < n; i++) {  
        if (i > 0) cout << " ";  
        cout << scores[i];  
    }  
    cout << endl;  
  
    cout << countPass(scores) << endl;
```

```
    cout << fixed << setprecision(1) << average(scores) << endl;

    return 0;
}
```

文件 ScoreAnalyzer.hpp :

```
#ifndef SCOREANALYZER_H
#define SCOREANALYZER_H

#include <vector>

using namespace std;

void sortDesc(vector<int>& scores);
int countPass(const vector<int>& scores);
double average(const vector<int>& scores);

#endif
```

25 程设II-周 17-课堂 1

类型：编程题

题面

最小栈

实现 MinStack：push/pop/top + 常数时间 getMin。主栈存所有元素，辅助栈存当前最小值（push 时压入 min(新元素, 辅助栈顶)，pop 时主栈辅助栈同时弹出），getMin 返回辅助栈顶。操作 1 val/2/3/4。

输入: 6 / 1 -2 / 1 0 / 1 -3 / 4 / 3 / 2

输出: -3 / -3

提交文件：MinStack.cpp

平台提供的初始文件

文件 main.cpp：

```
#include <iostream>
#include "MinStack.hpp"

int main() {
    MinStack minStack;
    int n;
    cin >> n;
```

```

    for (int i = 0; i < n; i++) {
        int op;
        cin >> op;
        if (op == 1) {
            int val;
            cin >> val;
            minStack.push(val);
        } else if (op == 2) {
            minStack.pop();
        } else if (op == 3) {
            cout << minStack.top() << endl;
        } else if (op == 4) {
            cout << minStack.getMin() << endl;
        }
    }

    return 0;
}

```

文件 MinStack.hpp :

```

#ifndef MINSTACK_H
#define MINSTACK_H

#include <stack>

using namespace std;

class MinStack {
private:
    stack<int> dataStack;
    stack<int> minStack;

public:
    MinStack();
    void push(int val);
    void pop();
    int top();
    int getMin();
};

#endif

```

25 程设II-周 17-课堂 2

类型：编程题

题面

STL 小工具：回文判断、排序、统计与 Top-K 查询

给定一行字符串和一组整数，完成：① stack 判回文 ② count_if 统计 > 阈值个数 ③ sort+lambda 降序 ④ accumulate 求乘积 ⑤ priority_queue 输出前 k 大。

输入：第一行字符串 text；第二行 n；第三行 n 个整数；第四行 threshold k。 **输出：** PALINDROME/NOT PALINDROME；>threshold 个数；降序数组；乘积；前 k 大。

输入: level / 5 / 1 10 20 15 30 / 15 3

输出: PALINDROME / 2 / 30 20 15 10 1 / 90000 / 30 20 15

提交文件：stl_practice.cpp

平台提供的初始文件

文件 main.cpp：

```
#include "stl_practice.hpp"
```

```
#include <iostream>
```

```
#include <vector>
```

```
void printVector(const std::vector<int>& nums) {  
    for (std::size_t i = 0; i < nums.size(); ++i) {  
        if (i > 0) std::cout << ' ';  
        std::cout << nums[i];  
    }  
    std::cout << '\n';  
}
```

```
int main() {  
    std::string text;  
    std::getline(std::cin, text);
```

```
    int n = 0;  
    std::cin >> n;
```

```
    std::vector<int> nums(n);  
    for (int i = 0; i < n; ++i) {  
        std::cin >> nums[i];  
    }
```

```
    int threshold = 0;  
    std::size_t k = 0;  
    std::cin >> threshold >> k;
```

```
    std::cout << (isPalindrome(text) ? "PALINDROME" : "NOT PALINDROME") <<  
    '\n';
```



```

std::cout << countGreaterThan(nums, threshold) << '\n';
printVector(sortDesc(nums));
std::cout << productAll(nums) << '\n';
printVector(topK(nums, k));

return 0;
}

```

文件 stl_practice.hpp :

```

#include <string>
#include <vector>
using namespace std;

bool isPalindrome(const string& text);
int countGreaterThan(const vector<int>& nums, int threshold);
vector<int> sortDesc(vector<int> nums);
long long productAll(const vector<int>& nums);
vector<int> topK(const vector<int>& nums, size_t k);

```

25 程设II-周 17-理论题（选择题，20 题）

注：平台尚未公布官方答案，以下为参考答案，建议结合课件核对。

第 1 题 stack、queue、priority_queue 属于哪一类？ A. 顺序容器 B. 关联容器 C. 容器适配器 D. 迭代器 **参考答案：C**

第 2 题 容器适配器的主要作用？ A. 完全独立于容器的新存储结构 B. 在顺序容器之上提供不同功能接口 C. 替代所有 STL 算法 D. 只用于文件 IO **参考答案：B**

第 3 题 stack 的元素访问特点？ A. FIFO B. 后进先出 LIFO C. 按关键字自动排序 D. 随机访问 **参考答案：B**

第 4 题 queue 的基本访问特点？ A. 头端插入尾端删除 B. 尾端插入头端删除 C. 任意位置插入删除 D. 总访问最大元素 **参考答案：B**

第 5 题 priority_queue 优先级默认由什么确定？ A. 元素类型的 < 操作 B. 插入时间 C. 下标 D. 地址大小 **参考答案：A**

第 6 题 stack 查看栈顶但不删除？ A. push() B. pop() C. top() D. front() **参考答案：C**

第 7 题 queue 的 front() 作用？ A. 返回队首元素 B. 返回队尾元素 C. 删除队首 D. 判空 **参考答案：A**

第 8 题 priority_queue 的 pop() 本质先调用哪类堆操作？ A. sort() B. find() C. pop_heap() D. count_if() **参考答案：C**

第 9 题 不指定底层容器时的默认情况？ A. stack 用 vector，queue 用 list B. stack 和 queue 默认 deque，priority_queue 默认 vector C. 三者都用 list D. 三者都用 map **参考答案：B**

第 10 题 stack 的底层容器必须支持哪些操作？ A. front()/pop_front()/insert() B. back()/push_back()/pop_back() C. find()/count()/erase() D. begin()/end()/sort() **参考答案：B**

第 11 题 c1.push(5); cout<<c1.size(); stack<int> c2(c1); cout<<c2.size(); deque<int> deq{3,1,4,1,5}; stack<int> c3(deq); cout<<c3.size(); A. 0 1 5 B. 1 1 5 C. 1 5 5 D. 5 1 1 **参考答案：B**

第 12 题 回文判断中，字符串长度为奇数时后半部分比较起点？ A. length/2 B. length/2 + 1 C. length-1 D. 0 **参考答案：B**（跳过正中间字符）

第 13 题 输入迭代器的主要功能？ A. 只写 B. 只读 C. 随机读写 D. 只能删除 **参考答案：B**

第 14 题 forward_list 的迭代器支持？ A. ++ 但不支持 -- B. -- 但不支持 ++ C. += 但不支持 ++ D. [] 随机访问 **参考答案：A**（单向链表，前向迭代器）

第 15 题 哪组容器通常提供随机访问迭代器？ A. array/vector/deque B. list/set/map C. forward_list/list/set D. map/multimap/set **参考答案：A**

第 16 题 C++11 范围 for 可用于？ A. 只有 map B. 只有 vector C. array、有 begin()/end() 的对象或花括号初始化列表 D. 只能用于普通数组 **参考答案：C**

第 17 题 关于迭代器失效，正确的是？ A. 只读方法一定使迭代器失效 B. 修改容器内容的方法可能导致迭代器或引用失效 C. 迭代器永不失效 D. 只有 cout 会导致失效 **参考答案：B**

第 18 题 for(auto it=v.begin(); it!=v.end(); it++){ if(*it==3) v.erase(it); } 主要问题？ A. erase() 返回字符串 B. erase() 后原迭代器可能失效，继续 it++ 有风险 C. vector 不能用 begin() D. if 不能判断整数 **参考答案：B**（正确写法 it = v.erase(it);）

第 19 题 sort(v.begin(),v.end(),[](int a,int b){return a>b;}); 第三个参数是？ A. 普通整数变量 B. 文件指针 C. Lambda 表达式 D. STL 容器 **参考答案：C**

第 20 题 为何工程项目不建议用 #include <bits/stdc++.h>？ A. 只能在 Python 用 B. 包含大量标准库内容，可能显著增加编译负担 C. 不能包含 STL 容器 D. 会自动删除源代码 **参考答案：B**
